

Linear containers for prompt/response systems: a note for the curious, not the initiated

MacBeth
for Neil

2026-08-22

Abstract

This is the short, plain version of `containers-over-vec.tex`. No proofs, almost no equations, everything landed on a prompt/response example. The technical note has the theorems; this note is so you can decide whether the technical note is worth your evening. One idea runs through all of it: *a response is a vector because a response is uncertain, and uncertainty is what a learning system trains*. Read it once, straight through.

1 The setup

Model a prompt as the simplest possible container: one request, and a space of possible replies. In symbols, $(\{*\}, A)$ — a single shape (the prompt) sitting over a *reply space* A . The only real choice is the word *space*. Why a vector space A and not just a set of allowed answers?

Because a reply is not one token. It is a distribution — a superposition over a basis of possible answers, carrying the model’s uncertainty. The basis is the vocabulary of possible replies; the coefficients are the confidence. And the basis in which that reply space is expressed is exactly what a learning system *learns*. So: responses = uncertainty = the basis of learning. That single sentence (yours) is the reason **Vec**, the category of vector spaces, is the right home rather than **Set**. Keep it in view; every section below is a consequence of it.

2 The structure of **Vec** we actually use

We lean on three facts about **Vec**, and one fact it is *missing*. One line each, each on the prompt example.

The zero object.

In **Vec** the empty reply space and the “full” terminal reply space are the *same* object, 0 . (In **Set** the empty set and the one-point set are wildly different; in **Vec** they collapse.) Harmless-looking, but it is the seed of everything.

The biproduct $A \oplus B$ — the cleanest hook.

Take prompt A (replies in A) and prompt B (replies in B). The space $A \oplus B$ is at once the *coproduct* — “I can answer *either* A or B ” — and the *product* — “I am ready for *both* A and B ”. Same space. In **Vec** “either” and “both” are literally the same object. In **Set** they come

apart (a tagged union $A \sqcup B$ versus a pair $A \times B$), and that is the whole difference between the two worlds.

Self-enrichment.

The maps between two reply spaces again form a reply space, and composing them is linear. This is what makes “linear container” more than a slogan and makes the computations go through. You can take it on faith here.

What *Vec* *lacks*: extensivity.

Set is extensive: a map into a union $A \sqcup B$ is a genuine partition of the domain — *you can always tell which branch of a choice you are in*. *Vec* is not extensive: a map into $A \oplus B$ is a pair of maps that may both be nonzero, so a vector $a + b$ carries no record of “which summand did I come from?” (Carboni–Lack–Walters). The disjoint union $\sqcup$ sits strictly inside the biproduct $\oplus$.

The punchline. That one missing property is the whole story. Because *Vec* cannot tell you which branch of a choice you are in, the shapes of a container go *invisible* (finitely many prompts with reply spaces of total dimension N become indistinguishable from N blank copies of the identity — the “biproduct collapse”), and the map that turns a container into its functor stops being *full* (there are more natural transformations than there are container morphisms, because a natural transformation may *mix* branches while a container morphism must *pick* one). Loss of shapes, loss of fullness: two faces of the one missing partition. Proofs are in `containers-over-vec.tex`; here I only want you to see that it is a single fact wearing two masks.

3 k -algebra and algebroid, seen through containers

Now: what does it mean to *compose* responses?

Within one prompt — a k -algebra. Stay with a single prompt A . Suppose you can take a response, run it, and feed its result back as another response of the *same* prompt, then read off one combined response. That is a map $A \otimes A \rightarrow A$ that combines two replies into one — associative, with a unit (the empty response). A reply space with such a combination law is exactly a k -algebra. So within a single prompt, the entire composition structure is: an algebra on its reply space. Nothing more is available, because there is only one prompt and nothing to route through.

Across prompts — an algebroid. Now allow several prompts and responses that carry *one* prompt into *another*. A response is no longer a self-loop; it has a source prompt and a target prompt. Over *Set* this “responses carry object to object, and compose” is the content of the theorem *directed containers = small categories*. The *Vec* version of a directed container is an *algebroid*: a k -algebra “with several objects”, i.e. a category whose hom-*sets* have been upgraded to hom-*spaces* (Bénabou/Mitchell). Prompts are the objects; between prompts a and b sits a whole reply space $P(a, b)$ of responses carrying a to b .

The right way to package this is a *Vec-matrix*: prompts on the rows and columns, entry $P(a, b)$ the reply space from a to b . Composition is then honest matrix multiplication,

$$(P \odot Q)(a, c) = \bigoplus_b P(a, b) \otimes Q(b, c),$$

“sum over every intermediate prompt b you could route through.” Its monoids are exactly algebroids: the multiplication is composition of responses, the unit picks the identity response at each prompt.

Why the naive version was not enough — and this ties back to extensivity. If you index reply data by a single prompt s (one space P_s per prompt), you can only ever express self-loops: response of s combining with response of s . You get a *family of one-object algebras*, one per prompt, never talking to each other. The reason is exactly the missing partition from §2: a bare index s has no room to record “*from a to b*” — source and target need two indices, not one. The cure is to put the $\oplus_b$ on the *composition* index (route through b) rather than on the shape index. That single relocated biproduct — the same $\oplus$ that made “either” equal “both” — is what upgrades a disconnected family of algebras into a genuine multi-object algebroid. The single-index theory is just the diagonal (self-loops only) of the matrix theory.

4 The wirings: a proposal

This is the part I actually want your eye on. Containers over **Vec** carry four monoidal products — $\oplus$, $\times$, $\otimes$, $\triangleleft$ — but in the finite linear setting two of them *coincide*, and that coincidence is itself the cleanest thing to say. Read as prompt/response *workflows*, what remains is *three* genuinely distinct ways to wire two systems together.

Choice and product are one wiring: $\oplus = \times$. Offer prompt A or prompt B . Over **Set** the menu (coproduct $A \sqcup B$) and the pair (product $A \times B$) are different constructions — a tagged union versus a table of both. Over **Vec** they are the *same* object, the biproduct $A \oplus B$ (§2): building the menu *is* being ready for both at once. So the two combinators a workflow language would ordinarily keep apart — “choose a branch” and “hold both branches” — collapse into one. This is not a defect to apologise for; it is the biproduct collapse of §2, now seen at the level of workflows. There is exactly one “combine by choice / product” operation, and it is $\oplus$. That leaves three:

$\oplus$ — **menu / both.**

Offer A or B ; by the biproduct this is the same as holding both reply spaces at once. Choice and product, a single wiring.

$\otimes$ — **parallel.**

Run two prompt/response systems side by side on independent inputs; the joint reply space is the tensor $A \otimes B$. Genuinely distinct from $\oplus$: a tensor *entangles* the two reply spaces where the biproduct keeps them additively separate.

$\triangleleft$ — **pipeline.**

Feed one system’s response in as the next system’s prompt. This is composition down a chain, and its bookkeeping is exactly the matrix product of §3: route the output prompt through to the next stage. (That operational reading — response-as-next-prompt — is the proposal layer, not a theorem.)

The **claim** is a clean slogan: *the algebra of wiring is the algebra of workflows*. Three categorical combinators — choose/hold, run-parallel, pipe — as the three connectives of a workflow language, with **Vec** the right home because the things being wired — responses — are *trainable vectors*, not inert tokens. The hope is that a workflow assembled this way stays differentiable in its

responses, so that learning can act on the whole wiring and not only on a single response; that would be the payoff of working over **Vec** rather than **Set**.

I want to be honest about altitude. This section is a *proposal*, a research direction, not a theorem. That the three wirings satisfy the laws you would want of a workflow calculus, that the pipeline reading of $\triangleleft$ is the right one, and that the linear setting genuinely buys trainability at the workflow level rather than only at the single-response level — all of that is what I would try to prove next. It is the “find a use” thrust, and it is speculative.

5 Status, honestly

Proved (all in `containers-over-vec.tex`):

- *Biproduct collapse*. Finitely many prompts with total reply-dimension N are indistinguishable, as a functor, from N blank copies of the identity; the shape set is not recoverable. Equivalently, “either A or B ” has reply space the biproduct $A \oplus B$, which is also “both”.
- *The extensivity crux*. An exact formula for maps between these functors shows the container-to-functor map is faithful but *not full*, and pinpoints the failure in the one symbol $\sqcup \subsetneq \oplus$ — the missing “which branch” partition.
- *$\triangleleft$ -comonoids = family of k -algebras*. The single-index composition gives exactly one algebra per prompt (self-loops), no cross-talk.
- *The correct home for algebroids*. Linear directed containers live in the matrix bicategory $\mathbf{Mat}(\mathbf{Vec})$; its (co)monoids are genuine algebroids (Bénabou/Mitchell), and the single-index theory is its diagonal.

Proposal, not yet proved:

- The three-wirings workflow calculus of §4 — that these combinators form a usable algebra of prompt/response workflows, and that **Vec** is the “useful” setting for prompt/response *learning*. This is the direction, not a result. (Note the honest side-benefit already banked: over **Vec** choice and product are the *same* wiring, $\oplus = \times$, so the calculus has three connectives, not four.)

That is the honest scorecard. The mathematics that is solid is elementary once you accept the base change; the excitement is that the one fact that breaks the representation theorem (non-extensivity) is exactly the fact an agent that must answer whatever it is asked would *want*. Whether that reading pays off as a real workflow calculus is the open bet.